

CLOUD COMPUTING

The Report :

Task 1: Deploy and access the Kubernetes Dashboard and JavaBenchmarkApp Docker Image

1.1 Setting the Kubernetes Dashboard

The Kubernetes Dashboard is set up by first adding its official Helm repository using **helm repo add**. The dashboard is then deployed into a dedicated namespace, ie; `kubernetes-dashboard`. To enable secure local access to the dashboard, `kubectl proxy` is used to create a temporary local connection to the Kubernetes API server. Also, port forwarding is configured with `kubectl port-forward` to map a local port (ie; 8443) to the dashboard's service port (ie; 443). The Dashboard is accessed at <https://localhost:8443>. A service account with all required administrative privileges is created and a token is generated for authentication which finally lets us in to the Dashboard.

1.2 Kubernetes Dashboard Implementation:

- **helm repo add kubernetes-dashboard https://kubernetes.github.io/dashboard/**
 - Adds the official Kubernetes Dashboard Helm repository to the local Helm configuration. This allows Helm to pull the Kubernetes Dashboard chart from the below repository.

```
Last login: Fri Sep 23 13:07:33 2022 from 192.168.0.1
student@cloud:~$ helm repo add kubernetes-dashboard https://kubernetes.github.io/dashboard/
helm repo add kubernetes-dashboard https://kubernetes.github.io/dashboard/

"kubernetes-dashboard" has been added to your repositories
student@cloud:~$
```

Figure 1.2.1

- **helm upgrade --install kubernetes-dashboard kubernetes-dashboard/kubernetes-dashboard --create-namespace --namespace kubernetes-dashboard**

Installs the Kubernetes Dashboard locally using Helm.

- `--install`: Installs the chart if it is not already installed.
- `--create-namespace`: Creates the `kubernetes-dashboard` namespace for future use.
- `--namespace=kubernetes-dashboard`: Deploys the Kubernetes into the aforementioned namespace

```

student@cloud:~$ helm upgrade --install kubernetes-dashboard kubernetes-dashboar
d/kubernetes-dashboard --create-namespace --namespace kubernetes-dashboard
Release "kubernetes-dashboard" does not exist. Installing it now.
NAME: kubernetes-dashboard
LAST DEPLOYED: Sat Dec 7 23:26:40 2024
NAMESPACE: kubernetes-dashboard
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
*****
*** PLEASE BE PATIENT: Kubernetes Dashboard may need a few minutes to get up and
  become ready ***
*****

Congratulations! You have just installed Kubernetes Dashboard in your cluster.

To access Dashboard run:
  kubectl -n kubernetes-dashboard port-forward svc/kubernetes-dashboard-kong-pro
xy 8443:443

NOTE: In case port-forward command does not work, make sure that kong service na
me is correct.
      Check the services in Kubernetes Dashboard namespace using:
      kubectl -n kubernetes-dashboard get svc

Dashboard will be available at:
  https://localhost:8443
student@cloud:~$

```

Figure 1.2.2

- **kubectl proxy**

- A proxy server that connects between local host & the Kubernetes Cluster's API Server.

```

student@cloud:~$ kubectl proxy
Starting to serve on 127.0.0.1:8001

```

Figure 1.2.3

- **kubectl -n kubernetes-dashboard port-forward svc/kubernetes-dashboard-kong-proxy 8443:443**

- This Step is used to access the Kubernetes Dashboard
- Port-forwards the service kubernetes-dashboard-kong-proxy in the kubernetes-dashboard namespace.
- Mapping the port so it can be accessed at <https://localhost:8443>.
- Figure 1.2.5 Shows the initial page of the Kubernetes Dashboard

```

student@cloud:~$
student@cloud:~$ kubectl -n kubernetes-dashboard port-forward svc/kubernetes-dashboard-kong-proxy 8443:443
Forwarding from 127.0.0.1:8443 → 8443
Forwarding from [::1]:8443 → 8443

```

Figure 1.2.4

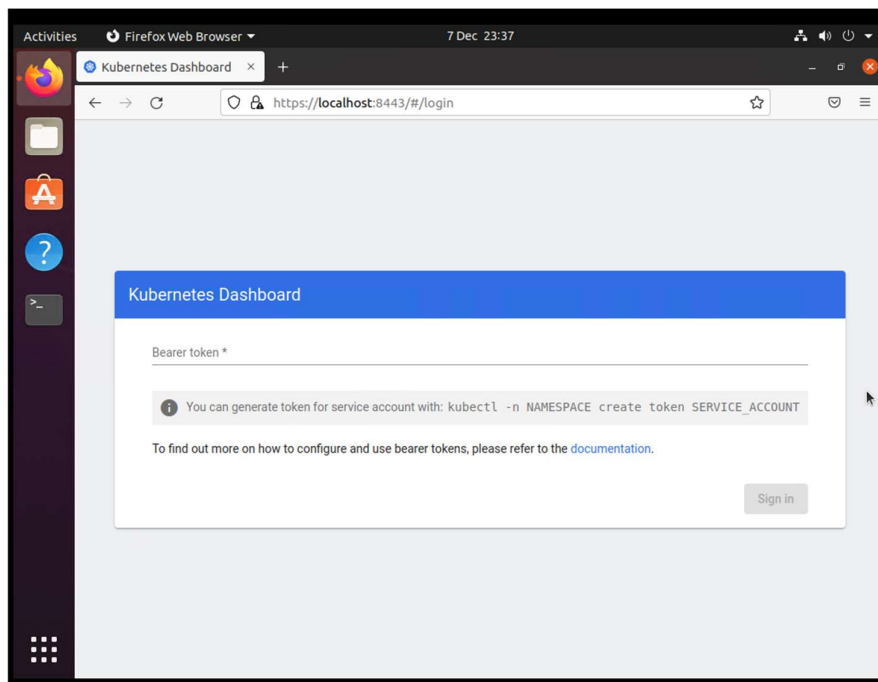


Figure 1.2.5

- **kubectl create serviceaccount dashboard-admin -n kubernetes-dashboard**
 - Creates a service account named dashboard-admin in the kubernetes-dashboard namespace.

```
student@cloud:~$ kubectl create serviceaccount dashboard-admin -n kubernetes-dashboard
serviceaccount/dashboard-admin created
student@cloud:~$
```

Figure 1.2.6

- **kubectl create clusterrolebinding dashboard-admin-binding --clusterrole=cluster-admin --serviceaccount=kubernetes-dashboard:dashboard-admin**
 - Creates a ClusterRoleBinding called dashboard-admin-binding.
 - Binds the cluster-admin role (with full cluster-wide capabilities) to the dashboard-admin service account in the kubernetes-dashboard namespace. This grants the administrator all the necessary permissions to access with the Kubernetes clusters.

```
student@cloud:~$ kubectl create clusterrolebinding dashboard-admin-binding \
> --clusterrole=cluster-admin \
> --serviceaccount=kubernetes-dashboard:dashboard-admin
clusterrolebinding.rbac.authorization.k8s.io/dashboard-admin-binding created
student@cloud:~$
```

Figure 1.2.7

- **kubectl -n kubernetes-dashboard create token dashboard-admin**
 - Generates a token for the dashboard-admin account in the kubernetes-dashboard namespace.
 - The token is used to log into the Kubernetes Dashboard
 - This finally gives access to the Dashboard as shown in

```
student@cloud:~$ kubectl -n kubernetes-dashboard create token dashboard-admin
eyJhbGciOiJSUzI1NiIsImtpZCI6ImtGTDBhQkRjc1o1bG54OHhEMGptWHB5SxkjcjAzTHlGNTFqb3VlU2dRTmMifQ.eyJhdWQiOi0lsiaHR0cHM6Ly9rdWJlcm5ldGVzLmRlZmF1bHQuc3ZjIl0sImV4cCI6MTczMzYxODY5NCwiaWF0IjoxNzMzMzNjE1MDk0LCJpc3MiOiJodHRwczovL2t1YmVybmV0ZXMuzGVmYXVsdC5zdmljLjZWFjY291bnQ6a3ViZXJlcy1kYXNoYm9hcmQ6ZGFzaGJvYXJkLWwvLnAATpwXId5Z0r0lDGF2JfnSKBnZAQY2iX--LdR9niSxnI0Ih2ag8R1c2iIvNl-9pi206pVuqWJQ36Yaiolnm1yBGtTG2mNG6qbNoxgIgKrmSi0Z-LctOX2QI0loaRB9Wst40k18asmpdmgEv-wR5dp0F60Z4GF0Q4R3NF7lRoKQhRpFRMdQwg
```

Figure 1.2.8

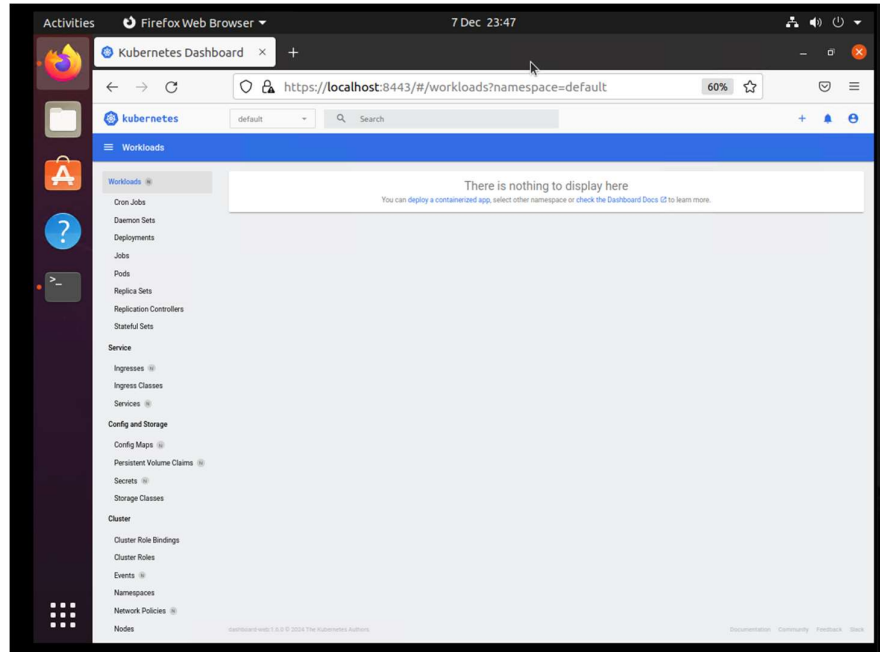


Figure 1.2.9

1.3 Deploying the Docker image "nclcloudcomputing/javabenchmarkapp" via CLI & Deploying a NodePort service

- **kubectl create deployment javabenchmarkapp --image=nclcloudcomputing/javabenchmarkapp --port=8080**

- Creates a Deployment named javabenchmarkapp from the image aforementioned.
- Exposes the application internally on port 8080.

```
student@cloud:~$ kubectl create deployment javabenchmarkapp --image=nclcloudcomputing/javabenchmarkapp --port=8080
deployment.apps/javabenchmarkapp created
student@cloud:~$
```

Figure 1.3.1

- **kubectl expose deployment javabenchmarkapp --type=NodePort --name=javabenchmarkapp-service --port=8080 --target-port=8080**

- Exposes the javabenchmarkapp deployment as a Kubernetes Service named javabenchmarkapp-service.

```
student@cloud:~$ kubectl expose deployment javabenchmarkapp --type=NodePort --name=javabenchmarkapp-service --port=8080 --target-port=8080
service/javabenchmarkapp-service exposed
student@cloud:~$
```

Figure 1.3.2

- **kubectl patch service javabenchmarkapp-service -p '{"spec": {"ports": [{"port": 8080, "targetPort": 8080, "nodePort": 30000}]}}'**

- Modifies the javabenchmarkapp-service service to assign it a specific nodePort value of 30000.
- nodePort services allow you to access the service externally using <NodeIP>:30000.

```
student@cloud:~$ kubectl patch service javabenchmarkapp-service -p '{"spec": {"ports": [{"port": 8080, "targetPort": 8080, "nodePort": 30000}]}}'
service/javabenchmarkapp-service patched
student@cloud:~$
```

Figure 1.3.3

- **kubectl get deployments**

- This shows that the javabenchmark app is up and running.

```
student@cloud:~$ kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
javabenchmarkapp    1/1     1             1           12m
student@cloud:~$
```

Figure 1.3.4

- **Accessing the image via local host**

- <https://localhost:30000/primecheck> is used to check the deployed image locally.

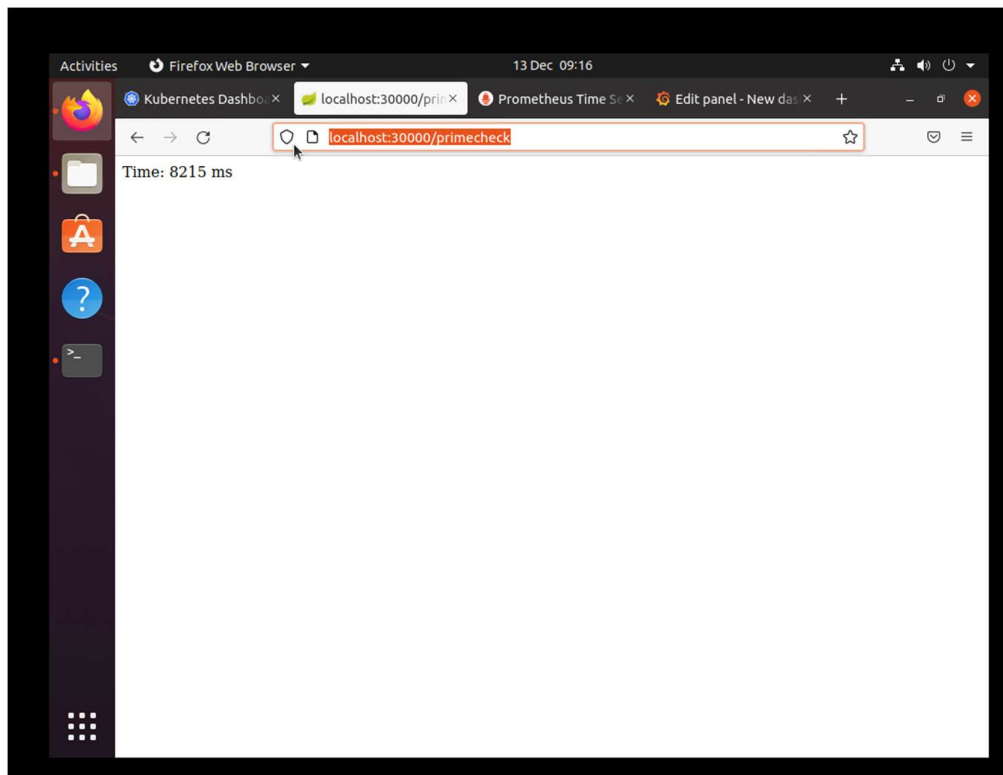


Figure 1.3.5

- Now, we can see the deployment in the Kubernetes dashboard too.

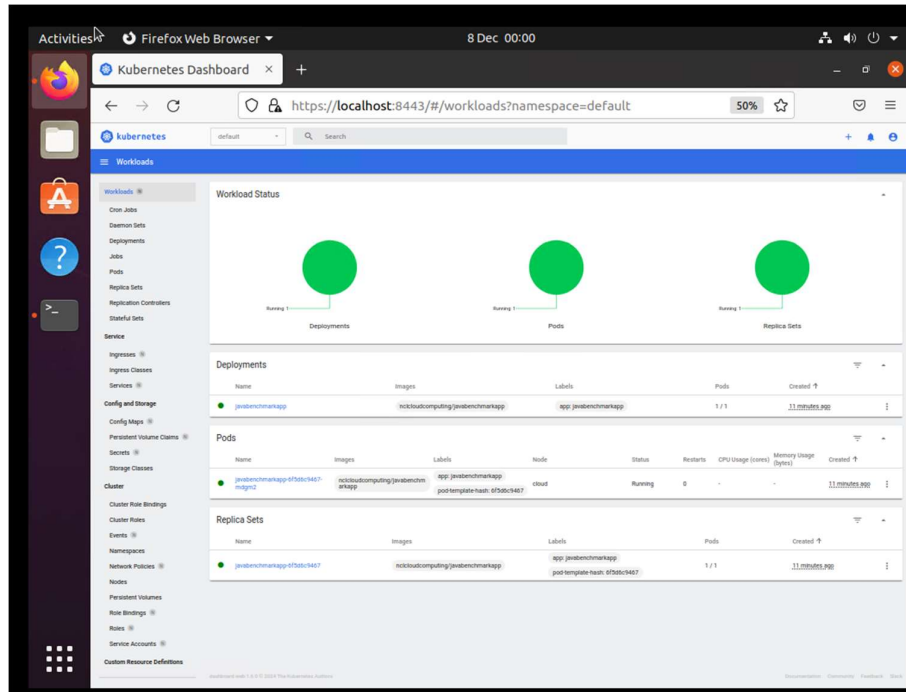


Figure 1.3.6

1.4 Additional Notes : The helm package manager was used instead of dashboard addon from microk8s.

Task 2: Deploy the monitoring stack of Kubernetes

2.1 Setting up a monitoring stack with Prometheus as Data Source with Grafana Visualisation Tool.

2.1.1 Installing the benchmarking tools

- **kubectl create namespace monitoringstack**

Creates a new namespace called monitoringstack for all the future benchmarking resources.

```
student@cloud:~$ kubectl create namespace monitoringstack
namespace/monitoringstack created
student@cloud:~$
```

Figure 2.1.1

- **helm repo add prometheus-community https://prometheus-community.github.io/helm-charts**

Adds the Prometheus Community Helm charts repository to Helm.


```
student@cloud:~$ helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
"prometheus-community" has been added to your repositories
student@cloud:~$
```

Figure 2.1.2

- **helm install prometheus prometheus-community/kube-prometheus-stack --namespace monitoringstack**

Installs Prometheus, Grafana, and all the related components in the monitoringstack namespace. This namespace will be used in further steps to benchmark the javacomputing workload.

```
student@cloud:~$ helm install prometheus prometheus-community/kube-prometheus-stack --namespace monitoringstack
NAME: prometheus
LAST DEPLOYED: Sun Dec  8 04:05:30 2024
NAMESPACE: monitoringstack
STATUS: deployed
REVISION: 1
NOTES:
kube-prometheus-stack has been installed. Check its status by running:
  kubectl --namespace monitoringstack get pods -l "release=prometheus"

Visit https://github.com/prometheus-operator/kube-prometheus for instructions on how to create & configure Alertmanager and Prometheus instances using the Operator.
student@cloud:~$
```

Figure 2.1.3

2.1.2 Accessing Prometheus

- **kubectl expose pod prometheus-prometheus-kube-prometheus-prometheus-0 --type=NodePort --name=prometheus-service --namespace monitoringstack --port=9090**

Using a NodePort, Now I am Exposing the Prometheus pod as a service named prometheus-service on port 9090. This now activates all the components in the kube-prometheus-stack. This includes the GUI dashboards for Prometheus and Grafana. This also sets the metric servers.

```
student@cloud:~$ kubectl expose pod prometheus-prometheus-kube-prometheus-prometheus-0 --type=NodePort --name=prometheus-service --namespace monitoringstack --port=9090
service/prometheus-service exposed
student@cloud:~$
```

Figure 2.1.4

- **kubectl get services -n monitoringstack**

Lists all services in the monitoringstack namespace.


```
student@cloud:~$ kubectl get services -n monitoringstack
```

NAME	AGE	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
prometheus-kube-prometheus-prometheus	4m45s	ClusterIP	10.152.183.106	<none>	9090/TCP,8080/TCP
prometheus-kube-prometheus-operator	4m45s	ClusterIP	10.152.183.42	<none>	443/TCP
prometheus-grafana	4m45s	ClusterIP	10.152.183.29	<none>	80/TCP
prometheus-kube-state-metrics	4m45s	ClusterIP	10.152.183.59	<none>	8080/TCP
prometheus-prometheus-node-exporter	4m45s	ClusterIP	10.152.183.18	<none>	9100/TCP
prometheus-kube-prometheus-alertmanager	4m45s	ClusterIP	10.152.183.123	<none>	9093/TCP,8080/TCP
alertmanager-operated	94/UDP 4m31s	ClusterIP	None	<none>	9093/TCP,9094/TCP,90
prometheus-operated	4m30s	ClusterIP	None	<none>	9090/TCP
prometheus-service	2m18s	NodePort	10.152.183.57	<none>	9090:31129/TCP

```
student@cloud:~$
```

Figure 2.1.5

The Figure above shows that all of the services Grafana, Prometheus, and the metrics server are deployed as a DaemonSet. We can now use the Prometheus-service port details to access the Prometheus dashboard. I.e; It can be locally accessed via <https://localhost:31129>.

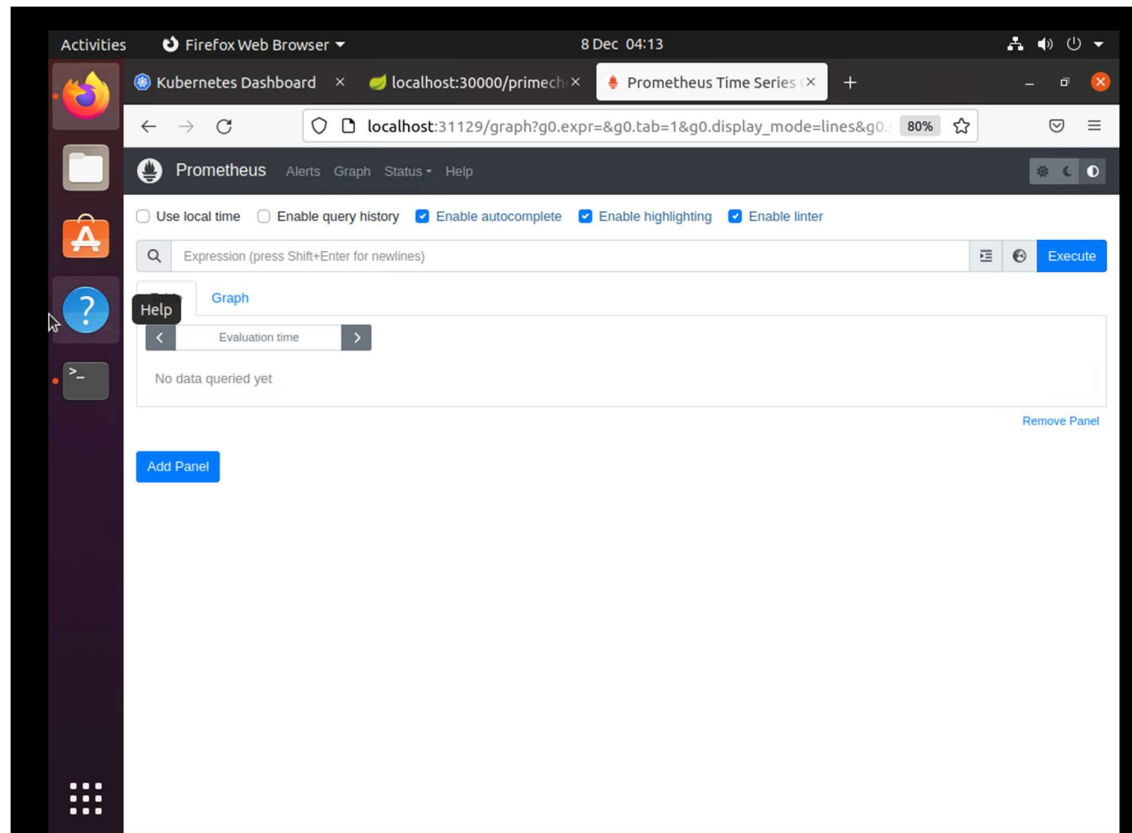


Figure 2.1.5.1

Now we can start setting up our Grafana component.

2.1.3 Accessing Grafana

- **kubectl get secret --namespace monitoring prometheus-grafana -o jsonpath="{.data.admin-password}" | base64 --decode ; echo**

Retrieves and decodes the admin password for Grafana , This will be used to log on to the grafana interface.

```
student@cloud:~$ kubectl get secret --namespace monitoring prometheus-grafana -o jsonpath="{.data.admin-password}" | base64 --decode ; echo
prom-operator
student@cloud:~$
```

Figure 2.1.6

- **kubectl port-forward svc/prometheus-grafana 3000:80 --namespace monitoringstack**

Forwards the local port 3000 to Grafana's service in the monitoringstack namespace, allowing access via <http://localhost:3000>.

```
student@cloud:~$ kubectl port-forward svc/prometheus-grafana 3000:80 --namespace monitoringstack
Forwarding from 127.0.0.1:3000 -> 3000
Forwarding from [::1]:3000 -> 3000
```

Figure 2.1.7

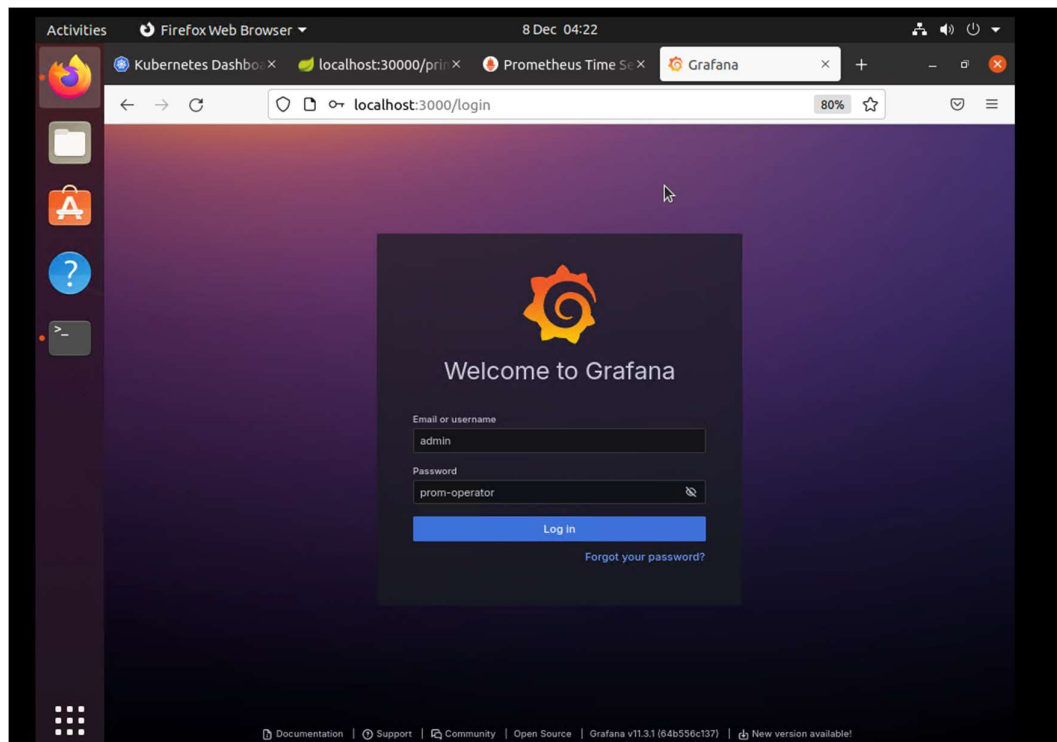


Figure 2.1.8

We can now use the credentials that we generated previously for Grafana to access its dashboard.

2.1.4 Verifying the components of benchmarking stack

- **kubectl get deployment -n monitoringstack**

Lists all benchmarking deployments in the monitoringstack namespace.

This Confirms all three crucial component mentioned in the

```
student@cloud:~$ kubectl get deployment -n monitoringstack
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
prometheus-kube-prometheus-operator 1/1      1             1            13m
prometheus-kube-state-metrics        1/1      1             1            13m
prometheus-grafana                   1/1      1             1            13m
student@cloud:~$
```

Figure 2.1.9

This shows that all three required components (ie; Prometheus, metrics server, Grafana Respectively) to benchmark javabenchmarkapp is deployed.

2.2 The Prometheus and Grafana Dashboards

2.2.1 The Prometheus Dashboard

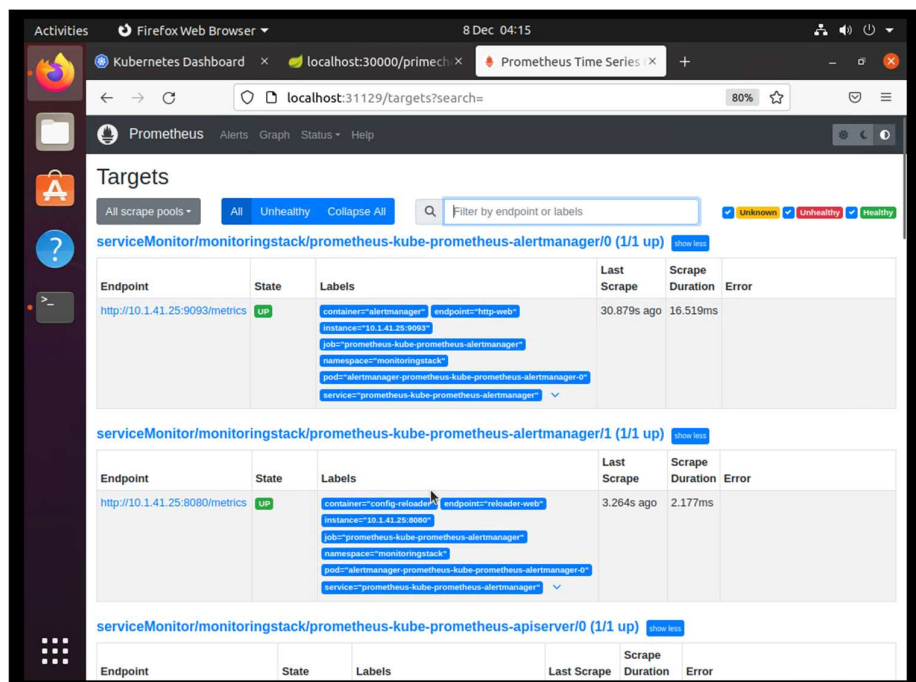


Figure 2.2.1

The above figure shows all the pods and services running properly.

2.2.2 The Grafana Dashboard

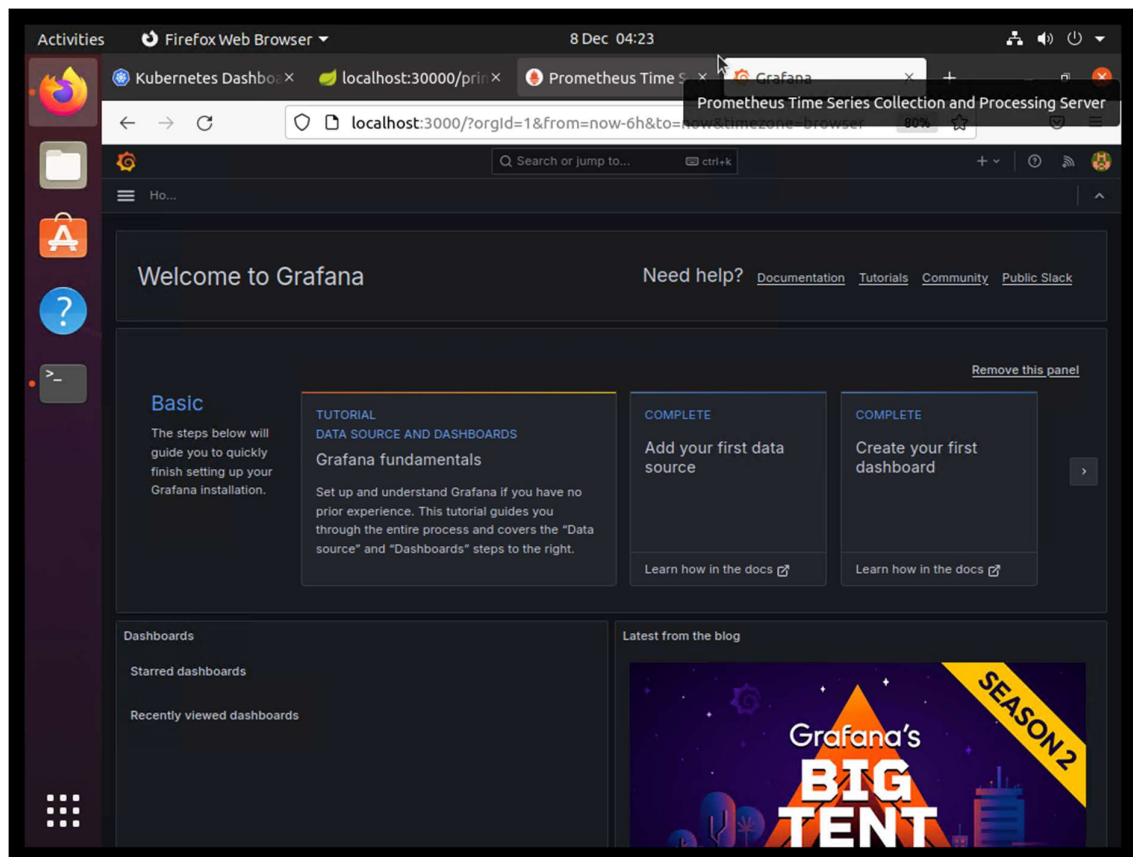


Figure 2.2.2

After authentication we can now access the Grafana Dashboard

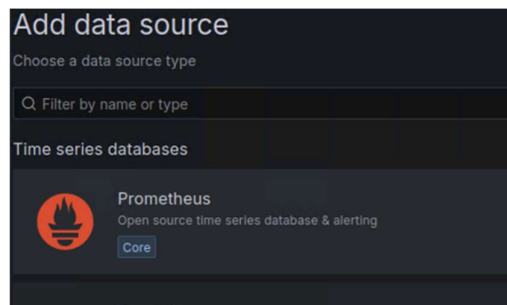


Figure 2.2.3

Now I have added Prometheus as the Data Source for visualization.

2.3 The Task Requirement Analysis

1. Grafana must reach Prometheus as a data source

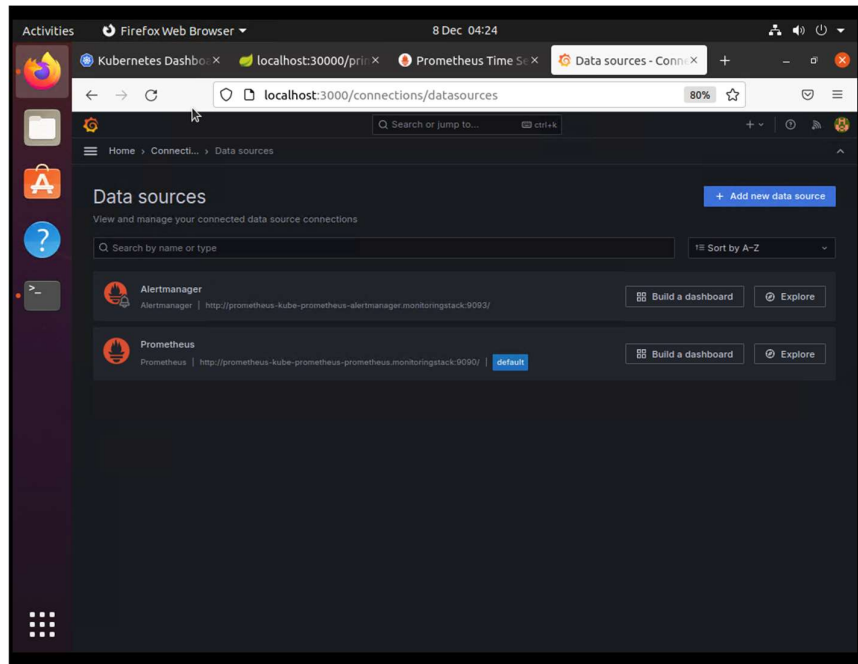


Figure 2.3.1

The above figure shows that Prometheus is already set up as a data source. This was done by selecting Prometheus as a data source in the previous step.

2. Grafana must be reachable from the host

We Accessed the dashboard by port forwarding it in the previous step.

```
student@cloud:~$ kubectl port-forward svc/prometheus-grafana 3000:80 --namespace monitoringstack
Forwarding from 127.0.0.1:3000 → 3000
Forwarding from [::1]:3000 → 3000
```

Figure 2.3.1

This ensures the Grafana Dashboard can be accessed locally through <http://localhost:3000>.

3. Prometheus must reach each instance of the metric server to collect metrics

This was set up while using helm package manager. If we look at the “Targets”/Running Components we can see that the Kube-state-metrics and Node-Exporter are running.

serviceMonitor/monitoringstack/prometheus-kube-state-metrics/0 (1/1 up) show less					
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://10.1.41.22:8080/metrics	UP	container="kube-state-metrics" endpoint="http" instance="10.1.41.22:8080" job="kube-state-metrics" namespace="monitoringstack" pod="prometheus-kube-state-metrics-6599589cbd-dt28f" service="prometheus-kube-state-metrics" ▾	28.365s ago	5.538ms	

serviceMonitor/monitoringstack/prometheus-prometheus-node-exporter/0 (1/1 up) show less					
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://192.168.0.100:9100/metrics	UP	container="node-exporter" endpoint="http-metrics" instance="192.168.0.100:9100" job="node-exporter" namespace="monitoringstack" pod="prometheus-prometheus-node-exporter-9jfc8" service="prometheus-prometheus-node-exporter" ▾	22.463s ago	33.936ms	

Figure 2.3.1

4. The metric server must be deployed as a DaemonSet

If we check our deployments in Figure 2.1.9 , we can see that the Kube-State-Metrics ie; The Metric Server is running as a Daemon set.

Task 3: Load Generator and deploying it as a Standalone Docker image

3.1 Creating a Load Generator in Python

3.1.1 The Design

The setup entails obtaining the target URL and request frequency from environment variables. A curl command is used to check connectivity and ensure that the destination server can be reached. The load generator then sends HTTP GET requests to the target, monitoring response times and failures. Multiple threads are started to submit requests at the appropriate frequency, simulating server load. Metrics such as average response time and failure count are presented during the procedure. The load generator ensures that all threads are properly terminated and terminates gracefully when interrupted. This Program Successfully stress tests the javabenchmarking app based on the arguments provided. This generator takes two parameters: the app's Target_Url and the frequency/number of requests that need to be sent. These characteristics are kept editable for modularity and flexible future use.

3.1.2 The Code

```
import os
import time
import subprocess
import requests
from threading import Thread

class PerformanceTester:
    """
    A program that tests the performance of JavaBenchmarkApp
    """

    def __init__(self, endpoint, rate):
        """
        Initializes the PerformanceTester with the target URL and request rate.

        Args:
            endpoint (str): The URL of the target endpoint to test.
            rate (int): The number of requests per second to send.
        """
        self.endpoint = endpoint
        self.rate = rate
        self.latencies = []
        self.error_count = 0
        self.stop_signal = False

    def check_connectivity(self):
        """
        Checks connectivity to the target endpoint using a curl command.
        Prints the response if successful or an error message if the test fails.
        """
        try:
            result = subprocess.run(['curl', '-s', self.endpoint], capture_output=True, text=True)
            if result.returncode == 0:
                print("Curl connection successful. Response:")
                print(result.stdout)
            else:
                print("Curl connection failed. Error:")
                print(result.stderr)
        except Exception as e:
            print(f"Error during curl test: {e}")

    def make_request(self):
        """
        Sends an HTTP GET request to the target endpoint.
        Measures latency and tracks errors based on the response status.
        """
        try:
            print(f"Sending request to {self.endpoint}...")
            response = requests.get(self.endpoint, timeout=20) # Timeout increased to 20 seconds for testing
            print(f"Response Status: {response.status_code}")
            print(f"Response Content: {response.text}")
```



```

if response.status_code == 200:
    response_text = response.text.strip()
    if response_text.startswith("Time:"):
        time_ms = int(response_text.split(":")[1].strip().replace("ms", ""))
        self.latencies.append(time_ms / 1000) # Convert ms to seconds
    else:
        print(f"Unexpected response format: {response_text}")
    else:
        print(f"Request failed with status code: {response.status_code}")
        self.error_count += 1
except requests.exceptions.RequestException as e:
    print(f"Request exception: {e}")
    self.error_count += 1

def begin(self):
    """
    Starts the performance test by running multiple threads to send requests concurrently.
    Performs a connectivity check before initiating the load.
    Listens for a keyboard interrupt to stop the Program
    """
    # Run the curl connectivity check before starting the request loop
    self.check_connectivity()

    def worker():
        while not self.stop_signal:
            self.make_request()
            time.sleep(1 / self.rate)

    threads = []
    for _ in range(self.rate):
        thread = Thread(target=worker)
        thread.start()
        threads.append(thread)

    try:
        # Keep running until interrupted
        while not self.stop_signal:
            time.sleep(1)
    except KeyboardInterrupt:
        print("\nKeyboard interrupt received. Stopping load generator...")
        self.stop_signal = True
    finally:
        for thread in threads:
            thread.join()

def generate_report(self):
    """
    Calculates the metrics average latency and displays the total error count ,
    prints it .
    """
    if self.latencies:
        average_latency = sum(self.latencies) / len(self.latencies)

```

```

else:
    average_latency = 0

print(f"Average Latency: {average_latency:.2f}s")
print(f"Error Count: {self.error_count}")

if __name__ == "__main__":
    """
    The Main Function
    Reads Target URL for target endpoint and frequency Of requests to give.
    Starts the load generation and generates a report upon completion.
    """

    # The parameters can be edited so it can be used for any deployments
    endpoint = os.getenv("TARGET", "http://192.168.0.100:30000/primecheck")
    rate = int(os.getenv("FREQUENCY", "10"))

    print(f"Starting performance test for {endpoint} at {rate} requests/second...")
    performance_tester = PerformanceTester(endpoint, rate)
    performance_tester.begin()
    performance_tester.generate_report()

```

3.1.3 The Algorithm

- **Step 1.** The `check_connectivity` method verifies connectivity to the endpoint using a `curl` command. The status of the connection will be printed to the console.
- **Step 2.** Then `makes_request` method sends HTTP GET requests to the target endpoint that is described. Measures response latency (in seconds) when the server provides a response in milliseconds.
- **Step 3.** The `begin` method runs multiple threads to send requests concurrently by using Step 1 to check the connectivity then Step 2 to send request concurrently. This will stop if interrupted in the console.
- **Step 4.** The `generate_report` method calculates and prints:
 - Average latency (time taken for requests).
 - Error count (number of requests that failed).

3.1.4 Explaining the Parameters

- The URL parameter "endpoint": This option provides the URL to which the program will send requests. It is retrieved by `os.getenv("TARGET", "http://192.168.0.100:30000/primecheck")`. The `os.getenv("TARGET")` function retrieves the value of the environment variable TARGET. If the URL is not set, the default is "http://192.168.0.100:30000/primecheck".
- The "rate" parameter determines how frequently an action or task is executed. It is retrieved by using `int(os.getenv("FREQUENCY", "10"))`. If FREQUENCY is not specified, it defaults to 10. This variable can be used to control how frequently the application makes requests to the deployed app.

3.1.5 Test running the Generator

```
python3 LoadGenerator.py
student@cloud:~$ python3 LoadGenerator.py
Starting performance test for http://192.168.0.100:30000/primecheck at 10 requests/second ...
Curl connection successful. Response:
Time: 4625 ms
Sending request to http://192.168.0.100:30000/primecheck ...
Sending request to http://192.168.0.100:30000/primecheck ...
Sending request to http://192.168.0.100:30000/primecheck ...
Sending request to http://192.168.0.100:30000/primecheck ...
Sending request to http://192.168.0.100:30000/primecheck ...
Sending request to http://192.168.0.100:30000/primecheck ...
Sending request to http://192.168.0.100:30000/primecheck ...
Sending request to http://192.168.0.100:30000/primecheck ...
Sending request to http://192.168.0.100:30000/primecheck ...
Response Status: 200
Response Content: Time: 8670 ms
Response Status: 200
Response Content: Time: 8790 ms
Response Status: 200
Response Content: Time: 8710 ms
Response Status: 200
Response Content: Time: 8684 ms
Response Status: 200
Response Content: Time: 8681 ms
Response Status: 200
Response Content: Time: 8766 ms
Response Status: 200
Response Content: Time: 8660 ms
Response Status: 200
Response Content: Time: 8861 ms
Sending request to http://192.168.0.100:30000/primecheck ...
Sending request to http://192.168.0.100:30000/primecheck ...
Sending request to http://192.168.0.100:30000/primecheck ...
Sending request to http://192.168.0.100:30000/primecheck ...
Sending request to http://192.168.0.100:30000/primecheck ...
Sending request to http://192.168.0.100:30000/primecheck ...
Sending request to http://192.168.0.100:30000/primecheck ...
Response Status: 200
Response Content: Time: 8879 ms
Sending request to http://192.168.0.100:30000/primecheck ...
Response Status: 200
Response Content: Time: 9408 ms
Sending request to http://192.168.0.100:30000/primecheck ...
Response Status: 200
Response Content: Time: 9063 ms
Response Status: 200
Response Content: Time: 9122 ms
Sending request to http://192.168.0.100:30000/primecheck ...
Sending request to http://192.168.0.100:30000/primecheck ...
Response Status: 200
Response Content: Time: 9187 ms
```

Figure 3.1.1

This shows that the load Generator is working properly as a standalone program. It does all the function functions mentions in the previous sections

```
Response Content: Time: 9948 ms
Average Latency: 9.80s
Error Count: 0
student@cloud:~$ █
```

Figure 3.1.2

Displays the log of report collected in runtime when stopped.

3.2 The DockerFile

3.2.1 The Design

A docker file is used to load the load Generator as a Docker file. This can implemented as shown below.

3.2.2 The Code

```
# This image is based on Python 3.9 with low resource overhead.
FROM python:3.9-slim

# This installs curl, for testing connectivity.
RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*

# This sets the directory where commands will be executed
WORKDIR /app

# This copies the 'LoadGenerator.py' file from the host machine to the '/app' directory

# Install necessary Python dependencies
RUN pip install requests

# Set the entry point for the container to run the LoadGenerator script
# This defines the command that will run when the container starts,
# in this case, it runs the Python script 'LoadGenerator.py' using Python 3.
ENTRYPOINT ["python3", "LoadGenerator.py"]
```

3.2.2 The Implementation

- **Build The docker file**
docker build -t load-generator

```
student@cloud:~$ docker build -t loadgenx .
Sending build context to Docker daemon 233.7MB
Step 1/6 : FROM python:3.9-slim
--> 473b33535d11e
Step 2/6 : RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*
--> Using cache
--> 3e26063be9d4
Step 3/6 : WORKDIR /app
--> Using cache
--> 03fcf8266850
Step 4/6 : COPY LoadGenerator.py .
--> d0ffe3134ab4
Step 5/6 : RUN pip install requests
--> Running in 25e701de7b1b
Collecting requests
  Downloading requests-2.32.3-py3-none-any.whl (64 kB)
----- 64.9/64.9 kB 3.8 MB/s eta 0:00:00
Collecting certifi>2017.4.17
  Downloading certifi-2024.8.30-py3-none-any.whl (157 kB)
----- 157.3/157.3 kB 2.1 MB/s eta 0:00:00
Collecting charset-normalizer>4, <2
  Downloading charset-normalizer-3.4.0-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (144 kB)
----- 144.9/144.9 kB 18.0 MB/s eta 0:00:00
Collecting idna>=2.5
  Downloading idna-3.10-py3-none-any.whl (70 kB)
----- 70.4/70.4 kB 4.8 MB/s eta 0:00:00
Collecting urllib3>=1.21.1
  Downloading urllib3-2.2.3-py3-none-any.whl (126 kB)
----- 126.3/126.3 kB 5.4 MB/s eta 0:00:00
Installing collected packages: urllib3, idna, charset-normalizer, certifi, requests
Successfully installed certifi-2024.8.30 charset-normalizer-3.4.0 idna-3.10 requests-2.32.3 urllib3-2.2.3
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommend
ed to use a virtual environment instead: https://pip.pypa.io/warnings/venv

[notice] A new release of pip is available: 23.0.1 -> 24.3.1
[notice] To update, run: pip install --upgrade pip
Removing intermediate container 25e701de7b1b
--> b8acde539d8d
Step 6/6 : ENTRYPOINT ["python3", "LoadGenerator.py"]
--> Running in 08d365c00719
Removing intermediate container 08d365c00719
--> 2ee69659ae58
Successfully built 2ee69659ae58
Successfully tagged loadgenx:latest
student@cloud:~$
```

Figure 3.2.1

- **Tag the docker file to call it later (Optional)**
docker tag load-generator localhost:32000/load-generator
- **Push the docker file as an image**
docker push localhost:32000/load-generator

```

student@cloud:~$ docker push localhost:32000/loadgenerator-x
Using default tag: latest
The push refers to repository [localhost:32000/loadgenerator-x]
ed9b51931eb9: Pushed
c34b6950ee87: Pushed
01375e6de53e: Mounted from load-generator
c07461a519e2: Mounted from load-generator
639cbe945cf9: Mounted from load-generator
13f3346e3d1d: Mounted from load-generator
0fd5a9e94c37: Mounted from load-generator
c0f1022b22a9: Mounted from load-generator
latest: digest: sha256:84de72aeaf9a4dc46be7f51fffe6aedd79a930626def52f6cbf523fa7cacee3b size: 1995
student@cloud:~$

```

Figure 3.2.2

Task 4: The Benchmarking

4. Benchmarking the Program

4.1 Deploying The Generator

4.1.1 The Deployment.yaml Code

```

apiVersion: apps/v1 # Kubernetes API version
kind: Deployment
metadata:
  name: load-generator-deployment # The name of the Deployment resource.
  namespace: monitoringstack # The Kubernetes namespace where this deployment will reside.
spec:
  replicas: 1 # Specifies that only one replica of the pod will be created.
  selector:
    matchLabels:
      app: load-generator
  template
    metadata: X
    labels: X
      app: load-generator
  spec:
    containers:
      - name: load-generator # The name of the container running inside the pod.
        image: localhost:32000/load-generator # The container image to use, replace with your image location.
        env: # Environment variables for configuring
          - name: TARGET # Environment variable that defines the target URL
            value: "http://192.168.0.100:30000/primecheck" # The URL where the load generator will send benchmark
requests.
          - name: FREQUENCY # Environment variable that defines the frequencY
            value: "11" # The number of requests per second the load generator will send.

```

This is the configuration file used to deploy and instantiate our program to the stack.

The command **kubectl apply -f deployment.yaml -n monitoringstack** is used to load the generator through below deployment.yaml file. This file contains the specifications for a deployment, including the number of replicas, the container image to be used, and the environment variables required such url and the frequency. By using the -n monitoringstack flag, the command ensures that these resources are applied to the monitoringstack namespace. This step sets up the load generator in the cluster, ensuring it runs as specified.

```
student@cloud:~$ kubectl apply -f deployment.yaml
deployment.apps/load-generator-deployment created
```

Figure 4.1.1

The command **kubectl get pods -n monitoringstack** allows you to list all the pods running in the monitoringstack namespace. This output provides details about the state of each pod, such as whether they are running, pending, or in an error state.

```
student@cloud:~$ kubectl get pods -n monitoringstack
NAME                                                    READY   STATUS    RESTARTS   AGE
prometheus-prometheus-node-exporter-9jfc6              1/1     Running   0           5d4h
prometheus-kube-prometheus-operator-5df7f7b5bb-58w4d  1/1     Running   0           5d4h
prometheus-kube-state-metrics-6598589cbd-dtz8f         1/1     Running   0           5d4h
alertmanager-prometheus-kube-prometheus-alertmanager-0 2/2     Running   0           5d4h
prometheus-grafana-7c5c795b7d-mr6wx                  3/3     Running   0           5d4h
prometheus-prometheus-kube-prometheus-prometheus-0    2/2     Running   0           5d4h
load-generator-deployment-677fc57b94-r5qd2             1/1     Running   0           5d3h
student@cloud:~$
```

Figure 4.1.2

All of our pods are running correctly.

We can see our deployment was succesfull by seeing the last pod.

Further check can be done by using **kubectl logs -n monitoringstack load-generator-deployment-677fc57b94-r5qd2-qcsdv**. This show the running of our generator program. This is a step for verifying that the deployment has successfully created and that the application is operational.

```
student@cloud:~$ kubectl logs -n monitoringstack load-generator-deployment-677fc57b94-r5qd2-qcsdv
Starting load generator for http://192.168.0.100:30000/primecheck at 10 requests/second...
Curl connection successful. Response:
Time: 1773 ms
Attempting request to http://192.168.0.100:30000/primecheck ...
Attempting request to http://192.168.0.100:30000/primecheck ...
Attempting request to http://192.168.0.100:30000/primecheck ...
Attempting request to http://192.168.0.100:30000/primecheck ...
Attempting request to http://192.168.0.100:30000/primecheck ...
Attempting request to http://192.168.0.100:30000/primecheck ...
Attempting request to http://192.168.0.100:30000/primecheck ...
Attempting request to http://192.168.0.100:30000/primecheck ...
Attempting request to http://192.168.0.100:30000/primecheck ...
Attempting request to http://192.168.0.100:30000/primecheck ...
Response Status: 200
Response Content: Time: 11896 ms
Attempting request to http://192.168.0.100:30000/primecheck ...
Response Status: 200
Response Content: Time: 12052 ms
Response Status: 200
Response Content: Time: 12076 ms
Response Status: 200
Response Content: Time: 12077 ms
Response Status: 200
```

Figure 4.1.3

4.2 Monitoring through Grafana

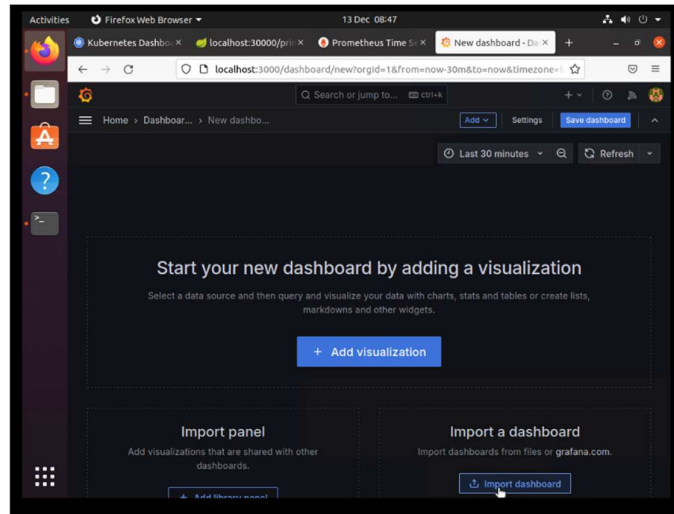


Figure 4.2.1

Add a new Dashboard in Grafana and add Prometheus as Data Source to scrape the metrics from the metric server we require to visualize the data

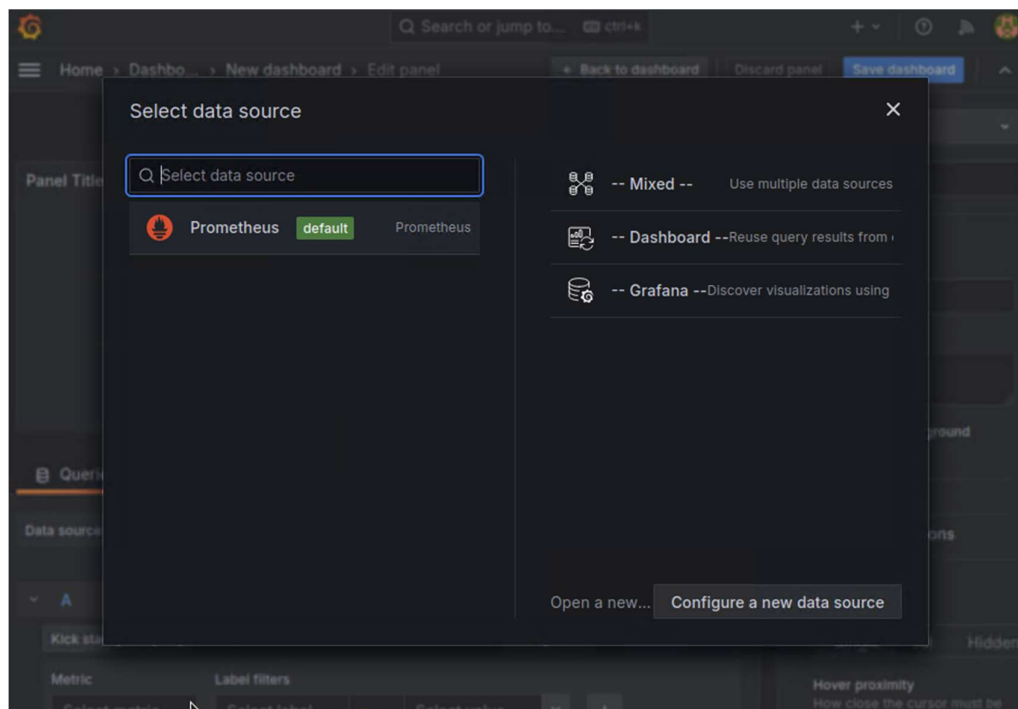


Figure 4.2.2

This links Prometheus as a data source hence we can access metric related to JavaBenchmarkapp.

To analyse data from grafana we can use queries . The coursework specifies to monitor CPU/memory usage of the web application so , we can use below codes.

Run the below Queries by adding a new query block to get the Result CPU Usage and Memory Usage of the JavaBenchmarkApp Program.

Query 1 : To plot CPU Usage

`Sum(container_memory_usage_bytes{pod=~javabenchmarkapp-6f5dc9467-fbltj"}) by(pod)`

This query calculates the memory usage in bytes for the specified pod (javabenchmarkapp-6f5dc9467-fbltj) and groups the result by pod name.

Query 2 : To plot Memory Usage

`sum by(pod) (rate(container_cpu_usage_seconds_total{pod=~"javabenchmarkapp-6f5dc9467-fbltj"}[2m]))`

This query calculates the CPU usage rate (in seconds) for the pod over a 2-minute interval and groups the result by pod name.

4.3 Benchmarking Results in Grafana :

4.3.1 The Memory Usage Graph Represented by Query 1

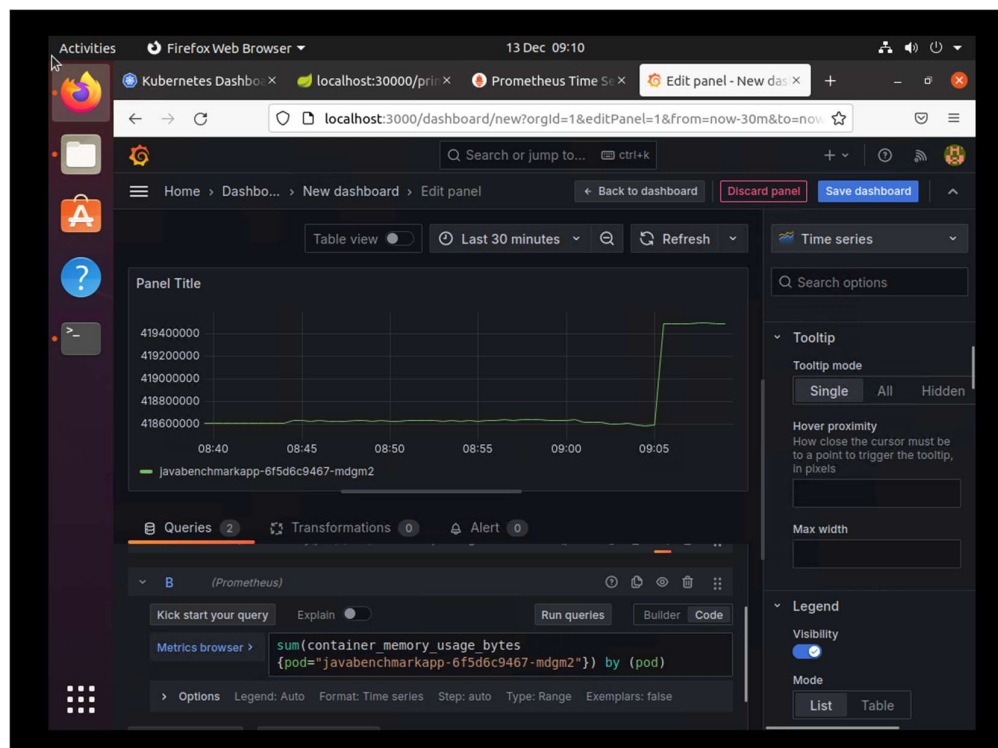


Figure 4.3.1

The memory usage of the app is suddenly incremented when the load generator is deployed.

4.3.2 The CPU Usage Graph Represented by query 2

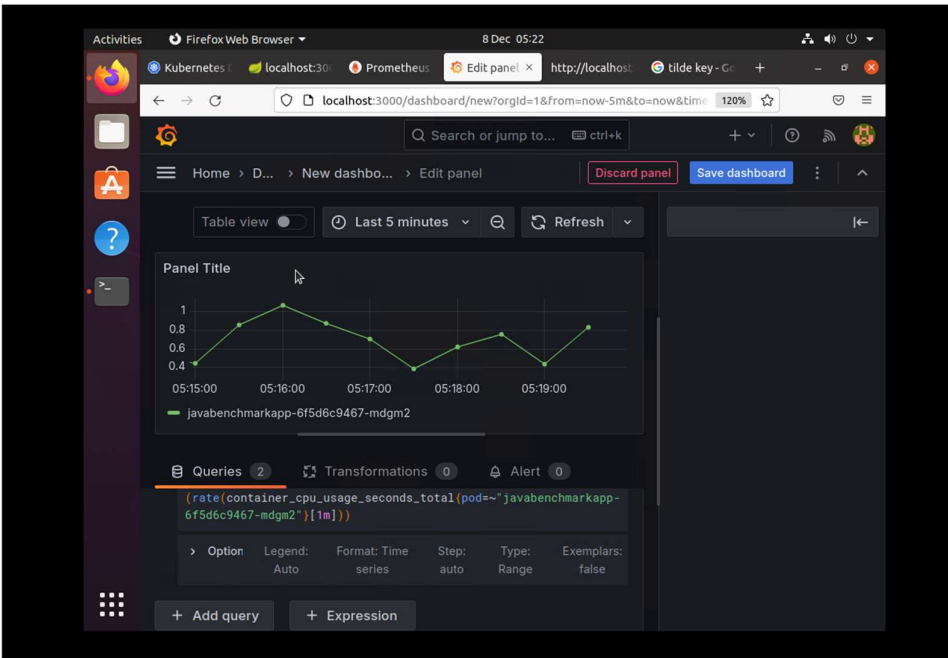


Figure 4.3.2

This short timeline graph explains how the cpu usage fluctuates with the stress that is given by the LoadGenerator.

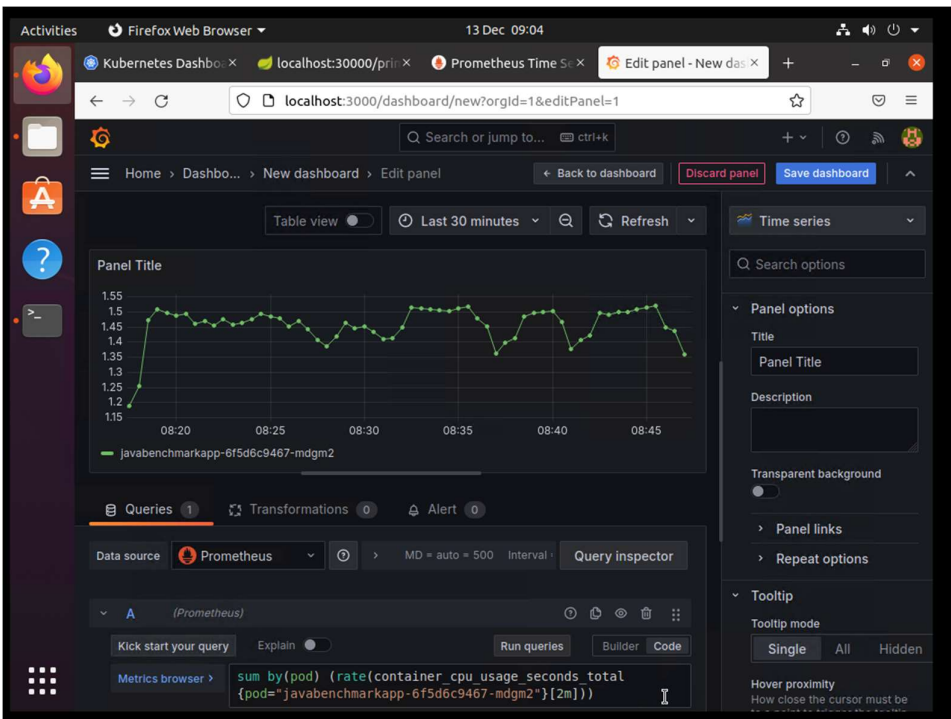


Figure 4.3.3

This trend can be observed in longer periods of timeline too. We sample data from past 30 minutes here.

4.4 The Results & Conclusion :

There is a direct correlation between the number of requests and the resources used by the deployed application:

- **When requests are high, resource usage (CPU and memory) increases significantly.**
- **When requests fail due to overload, resource usage drops.**

During peak loads, CPU and memory utilization are at their highest, reflecting the system's effort to handle the increased number of requests.

By leveraging Prometheus and Grafana, we can effectively monitor, analyze, and visualize resource usage trends for the JavaBenchmarkApp, enabling better performance benchmarking and system optimization.

This Hence Concludes the coursework.